Semiring-Based Linear Algebra Framework for Two-Way Regular Path Queries

Georgiy Belyanin[1][0009−0004−7527−4281], Rodion Suvorov[1][0009−00007−4934−4911], Iakov Kirilenko[1][0000−0003−4384−8274], and Semyon Grigoriev[1][0000−0002−7966−0698]

Saint Petersburg State University, Saint Petersburg, Russia belyaningeorge@ya.ru, suvorov.53245324@gmail.com, y.kirilenko@spbu.ru, s.v.grigoriev@mail.spbu.ru

Abstract. Two-way regular path queries (2-RPQs) allow the use of regular languages over edges and their inverses in edge-labeled graphs to constrain paths of interest. Adopted in modern graph query languages such as GQL, SQL/PGQ, and Cypher, 2-RPQs have become part of the GQL ISO standard. However, their efficient evaluation on large-scale graphs remains challenging. In this paper, we propose a semiring-based generalization of the linear algebra approach for evaluating single-source 2-RPQs. The original LARPQ algorithm solves the reachability problem using Boolean matrices. We generalize it by replacing Boolean algebra with arbitrary semirings, enabling a unified solution for various path-aggregation problems. Specifically, we employ the path semiring to find all simple paths and all trails. The algorithms are implemented using the SuiteSparse:GraphBLAS library within the LAGraph infrastructure. Experimental evaluation on real-world knowledge graphs demonstrates significant performance improvements over state-of-the-art solutions.

Keywords: Regular path queries · Two-way RPQ · Linear algebra · Semirings · Graph databases · Sparse matrices.

1 Introduction

Language-constrained path querying (FLPQ) [5] provides a mechanism for searching paths in edge-labeled graphs where constraints are specified in terms of formal languages. A path is considered valid if the word formed by concatenating the labels of its edges belongs to the specified language. When constraints are regular languages, such queries are called regular path queries (RPQs). In practice, it is often useful to allow traversal opposite to edge directions, leading to two-way regular path queries (2-RPQs) [10, 7]. Queries specifying only a start or an end vertex are known as single-source or single-destination queries, respectively. Both have been adopted in graph query languages such as GQL [14], SQL/PGQ, and SPARQL [1].

Graph database systems support different path query semantics, as the choice significantly affects both result interpretation and computational complexity.

Common semantics include: reachability (all vertices reachable via any valid path), all-shortest-paths (one shortest path per reachable vertex), all-simple-paths (all simple paths satisfying the query), and all-trails (all trails satisfying the query) [13]. Each semantics serves different use cases, and the query engine must select the appropriate algorithm accordingly.

Efficient evaluation of RPQs and 2-RPQs remains an active research area [11, 20]. Sparse linear algebra-based methods have shown promise for RPQ evaluation [3]. In particular, the LARPQ (Linear Algebra-based Regular Path Query) algorithm [6] employs a breadth-first traversal that simultaneously processes both the graph and the finite automaton representing the regular language. While LARPQ demonstrates promising performance, it was designed solely for reachability, and its application to path-related semantics (e.g., simple or shortest paths) remains unexplored.

The contributions of this paper are as follows.

- A formal generalization of the LARPQ algorithm to a semiring-based framework, enabling the solution of various path-aggregation problems within a unified algorithmic structure.
- Instantiation of the framework with specific semirings to solve distinct problems: Boolean semiring for reachability, and different path semirings with index unary operators for all-simple-paths and all-trails and all-shortest-paths semantics.
- Experimental evaluation on real-world knowledge graphs demonstrating significant performance improvements over state-of-the-art alternatives.

2 Preliminaries

In this section, we introduce the foundational concepts required throughout the paper, including edge-labeled graphs, paths, finite automata, and regular path queries.

2.1 Edge-Labeled Graphs

We begin by defining the graph data model used in this work.

Definition 1 (Edge-labeled graph). *A quadruple $G = \langle V, E, L, \lambda_G \rangle$ is called an edge-labeled graph (or simply a graph) if:*

- *V is a finite set of vertices;*
- *$E \subseteq V \times V$ is a finite set of edges;*
- *L is a finite set of labels;*
- *$\lambda_G : E \mapsto 2^L$ represents edge labels.*

For the remainder of this paper, we assume that vertices of any graph $G = \langle V, E, L, \lambda_G \rangle$ are enumerated, and without loss of generality, $V = \{1, 2, \ldots, |V|\}$.

To enable traversal in both directions along edges, we introduce the notion of inverted edges and labels. Let $G = \langle V, E, L, \lambda_G \rangle$ be an edge-labeled graph. We define:

- $a^- \notin L$ for each $a \in L$, with $(a^-)^- = a$ and $a = b \Leftrightarrow a^- = b^-$;
- $e^- = (v, u)$ where $e = (u, v)$;
- $E^- = \{e^- \mid e \in E\}$;
- $L^- = \{a^- \mid a \in L\}$;
- $\lambda_G^- : E^- \mapsto 2^{L^-}$, where $\lambda_G^-(e^-) = \{a^- \mid a \in \lambda_G(e)\}$.

We also define the bidirectional edge and label sets:

- $E^{\leftrightarrow} = E \cup E^-$;
- $L^{\leftrightarrow} = L \cup L^-$;
- $\lambda_G^{\leftrightarrow} : E^{\leftrightarrow} \mapsto 2^{L^{\leftrightarrow}}$, where $\lambda_G^{\leftrightarrow}(e) = \lambda_G(e) \cup \lambda_G^-(e)$;
- $G^{\leftrightarrow} = \langle V, E^{\leftrightarrow}, L^{\leftrightarrow}, \lambda_G^{\leftrightarrow} \rangle$.

RS

is $\lambda_G(e) \cup \lambda_G^-(e)$ defined correctly for both original and inverted edges?

The graph $G^{\leftrightarrow}$ allows traversal along both original and inverted edges, which is essential for evaluating two-way regular path queries.

2.2 Paths

Definition 2 (Path). *A path $\pi = (e_1, e_2, \ldots, e_n)$ of length n in the graph $G = \langle V, E, L, \lambda_G \rangle$ from u_1 to v_n is a finite sequence of edges $e_i = (u_i, v_i) \in E$ such that $\forall 1 \leq i \leq n-1 : v_i = u_{i+1}$.*

We denote $|\pi|$ as the length of a path π and consider zero-length paths, represented by an empty sequence from a vertex to itself.

Definition 3 (Simple path). *A path $\pi = (e_1, e_2, \ldots, e_n)$ is simple if it contains no repeated vertices. Formally, assuming $e_i = (v_i, u_i)$, this means that all vertices in the sequence $v_1, u_1, u_2, \ldots, u_n$ are pairwise distinct.*

Definition 4 (Trail). *A path $\pi = (e_1, e_2, \ldots, e_n)$ is a trail if it contains no repeated edges. Formally, $\forall 1 \leq i, j \leq n : e_i = e_j \Rightarrow i = j$.*

Definition 5 (Two-way path). *A two-way path (2-path) $\pi = (e_1, e_2, \ldots, e_n)$ in G is a path in $G^{\leftrightarrow}$. Two-way simple paths and two-way trails are defined analogously.*

The mapping ω associates paths with words over the label alphabet:

- $\omega_G(\pi) = \{a_1 \cdot a_2 \cdot \ldots \cdot a_n \mid a_i \in \lambda_G(e_i)\}$ for a path $\pi = (e_1, e_2, \ldots, e_n)$ in G;
- $\omega_{G^{\leftrightarrow}}(\pi) = \{a_1 \cdot a_2 \cdot \ldots \cdot a_n \mid a_i \in \lambda_G^{\leftrightarrow}(e_i)\}$ for a 2-path $\pi = (e_1, e_2, \ldots, e_n)$ in G.

Here, $\cdot$ denotes string concatenation.

2.3 Two-Way Finite Automata

Regular languages, recognized by finite automata, provide the foundation for specifying path constraints. For two-way path queries, we require a variant of nondeterministic finite automata.

Definition 6 (Two-way NFA). *A two-way nondeterministic finite automaton (2-NFA) is a tuple $N = \langle Q, \Sigma, \Delta, \lambda_N, Q_S, Q_F \rangle$, where:*

- *Q is a finite set of states;*
- *Σ is a finite alphabet;*
- *$\Delta \subseteq Q \times Q$ is the transition relation;*
- *$\lambda_N : \Delta \mapsto 2^{\Sigma^{\leftrightarrow}}$ maps transitions to labels;*
- *$Q_S \subseteq Q$ is the set of starting states;*
- *$Q_F \subseteq Q$ is the set of final states.*

Definition 7 (Two-way DFA). *A two-way deterministic finite automaton (2-DFA) is a special case of 2-NFA with $|Q_S| = 1$ and $\forall (q, q'), (q, q'') \in \Delta : q' \neq q'' \Rightarrow \lambda_N((q, q')) \cap \lambda_N((q, q'')) = \varnothing$.*

A 2-NFA can be viewed as a graph $G_N = \langle Q, \Delta, \Sigma^{\leftrightarrow}, \lambda_N \rangle$ equipped with sets of starting and final states. Conversely, a graph $G = \langle V, E, L, \lambda_G \rangle$ equipped with $V_S \subseteq V$ can be seen as a 2-NFA $N_G = \langle V, L, E, \lambda_G^{\leftrightarrow}, V_S, V \rangle$ that accepts:

$$[[N_G]] = \bigcup_{\text{2-path } \pi_G \text{ in } G \text{ from } v_S \in V_S} \omega_{G^{\leftrightarrow}}(\pi_G)$$

2.4 Two-Way Regular Path Queries

Definition 8 (Two-way regular path query). *A two-way regular path query (2-RPQ) is a triple $\langle G, R, v_s \rangle$ where $G = \langle V, E, L, \lambda_G \rangle$ is a graph, R is a regular language over the alphabet $L^{\leftrightarrow}$, and $v_s \in V$ is a starting vertex.*

We define several semantic variants of 2-RPQs that are considered in this paper:

- **S**ingle-**s**ource **s**hortest **p**aths (**SSSP**):

  $$[[Q]]_{SSSP} = \{\pi \mid \pi \text{ is a 2-simple path in } G \text{ from } v_s \text{ and } \omega_{G^{\leftrightarrow}}(\pi) \cap R \neq \varnothing\}.$$

RS
defined as simple path!!!

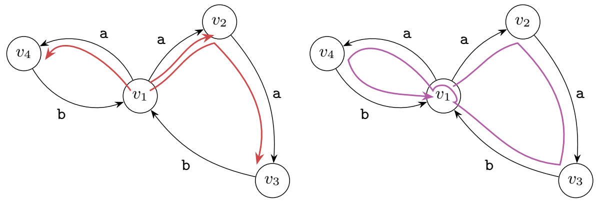
- **S**ingle-**d**estination **s**hortest **p**aths (**SDSP**):

  $$[[Q]]_{SDSP} = \{\pi \mid \pi \text{ is a 2-simple path in } G \text{ to } v_s \text{ and } \omega_{G^{\leftrightarrow}}(\pi) \cap R \neq \varnothing\}.$$

RS
defined as simple path!!!

- **S**ingle-**s**ource **a**ll **s**imple **p**aths (**SSASP**):

  $$[[Q]]_{SSASP} = \{\pi \mid \pi \text{ is a 2-simple path in } G \text{ from } v_s \text{ and } \omega_{G^{\leftrightarrow}}(\pi) \cap R \neq \varnothing\}.$$



(a) All simple paths (b) All trails (one of the longest paths) (c) All shortest paths (from v_1 to v_1)

Fig. 1: Different query semantics for $((a^+)(b?))^+$

- **S**ingle-**d**estination **a**ll **s**imple **p**aths (**SDASP**):

$$[[Q]]_{SDASP} = \{\pi \mid \pi \text{ is a 2-simple path in } G \text{ to } v_s \text{ and } \omega_{G^\leftrightarrow}(\pi) \cap R \neq \varnothing\}.$$

- **S**ingle-**s**ource **a**ll **t**rails (**SSAT**):

$$[[Q]]_{SSAT} = \{\pi \mid \pi \text{ is a 2-trail in } G \text{ from } v_s \text{ and } \omega_{G^\leftrightarrow}(\pi) \cap R \neq \varnothing\}.$$

- **S**ingle-**d**estination **a**ll **t**rails (**SDAT**):

$$[[Q]]_{SDAT} = \{\pi \mid \pi \text{ is a 2-trail in } G \text{ to } v_s \text{ and } \omega_{G^\leftrightarrow}(\pi) \cap R \neq \varnothing\}.$$

Figure 1 presents examples of single-source query results under different semantics on the same graph: **a)** *all simple paths*, **b)** *all trails*, and **c)** *all shortest paths*.

3 Linear Algebra, Graphs and Relations

In this section, we establish the linear algebra foundations required for our approach. We introduce the concept of semirings, discuss matrix representations of relations, and present the original LARPQ algorithm for solving the reachability problem.

3.1 Semirings

Semirings provide an algebraic framework that generalizes both Boolean algebra and path-based computations. A semiring consists of a set equipped with two binary operations that satisfy certain axioms.

Definition 9 (Semiring). *A semiring is a tuple* $\langle D_{out}, D_{in_1}, D_{in_2}, \oplus, \otimes, \mathbf{0}\rangle$ *where:*

- D_{out}, D_{in_1}, *and* D_{in_2} *are three domains;*
- $\langle D_{out}, \oplus, \mathbf{0}\rangle$ *is a commutative monoid with an identity element* $\mathbf{0} \in D_{out}$*;*
- $\langle D_{out}, D_{in_1}, D_{in_2}, \otimes\rangle$ *is a closed binary operator;*

In graph processing algorithms, semirings are defined differently from algebraic semirings due to the versatility of having different domains and irrelevance of distributivity and multiplicative identity elements. Two specific semirings are particularly important for this work: the Boolean semiring for reachability and the path semiring for enumerating paths.

3.2 Matrix Representations

Relations between finite sets can be represented using matrices. For two enumerated finite sets $A = \{1, 2, \ldots, n\}$ and $B = \{1, 2, \ldots, m\}$, a binary matrix T of size $n \times m$ can represent a relation $T \subseteq A \times B$ where $T_{ij} = 1 \Leftrightarrow (i, j) \in T$.

Let $G = \langle V, E, L, \lambda_G \rangle$ be an arbitrary graph and $N = \langle Q, \Sigma, \Delta, \lambda_N, Q_S, Q_F \rangle$ an arbitrary 2-NFA. For each label $a \in L^{\leftrightarrow}$, we define:

- $E_a = \{e \mid e \in E, a \in \lambda_G^{\leftrightarrow}(e)\}$;
- $\Delta_a = \{\delta \mid \delta \in \Delta, a \in \lambda_N(\delta)\}$.

Definition 10 (Adjacency matrix of a label). *Let G_a be the matrix representing the binary relation E_a. Then G_a is called the adjacency matrix of label a.*

Definition 11 (Boolean decomposition). *A Boolean decomposition of the adjacency matrix G is the set of Boolean matrices $\{G_a \mid a \in L^{\leftrightarrow}\}$.*

3.3 The Path Semiring

For solving path enumeration problems (all simple paths, all trails), we require a more expressive semiring based on path representations.

Let $\mathrm{R}_{Paths} = 2^{\mathbb{N}^*}$, where $\pi' = (v_1, v_2, \ldots, v_n) \in \mathbb{N}^*$ represents a 2-path $\pi = ((v_1, v_2), (v_2, v_3), \ldots, (v_{n-1}, v_n))$ of length $n - 1$. For $a, b \in \mathrm{R}_{Paths}$ and $c \in \mathbb{B}$, we define:

- **Sum:** $a \oplus b = a \cup b$;
- **Product:** $a \otimes c = \begin{cases} a & \text{if } c = 1 \\ \varnothing & \text{otherwise} \end{cases}$.

To this end, we can introduce a semiring for deciding more complex path problems:

- $\textsc{Paths} = \langle \mathrm{R}_{Paths}, \mathrm{R}_{Paths}, \mathbb{B} = \{0, 1\}, \cup, \mathbf{fst}, \varnothing \rangle$ where $\mathbf{fst}(x, y) = \begin{cases} x & \text{if } y = 1 \\ \varnothing & \text{otherwise} \end{cases}$

Let $A = (a_{ij})$, $B = (b_{ij})$ be matrices over R_{Paths} and $C = (c_{ij})$ be a Boolean matrix. We extend the operations to matrices:

- **Sum:** $A_{n \times m} \oplus B_{n \times m} = (a_{ij} \oplus b_{ij})_{n \times m}$;
- **Product:** $A_{n \times m} \otimes C_{m \times k} = (\bigoplus_{l=1}^{m} a_{il} \otimes c_{lj})_{n \times k}$;
- **Zero-matrix:** $0_{n \times k}^{\textsc{Paths}} = (\varnothing)_{n \times k}$.

3.4 Index Unary Operators

The path semiring alone is insufficient for enforcing path semantics (simple paths, trails). We introduce index unary operators that can filter paths based on vertex or edge repetition.

Definition 12 (Index unary operator). *Let $A_{n\times m} = (a_{ij})$ be a matrix over a semiring R. An index unary operator $I : R \times \mathbb{N} \times \mathbb{N} \mapsto R$ is applied element-wise: $I(A) = (I(a_{ij}, i, j))_{n\times m}$.*

For matrices over R_{Paths}, we define specific index unary operators:

- $I_{Simple}(a, i, j) = \{(v_1, v_2, \ldots, v_n, j) \mid (v_1, v_2, \ldots, v_n) \in a, v_1 \neq v_n$ and $\forall t : 2 \leq t \leq n : v_t \neq j\}$;
- $I_{Trail}(a, i, j) = \{(v_1, v_2, \ldots, v_n, j) \mid (v_1, v_2, \ldots, v_n) \in a$ and $\forall t : 1 \leq t \leq n-1 : (v_t, v_{t+1}) \neq (v_n, j)\}$.

RS

dobule check there: simple should filter out cycles

The I_{Simple} operator extends paths while filtering out those that would create vertex repetitions (enforcing simple paths). The I_{Trail} operator filters out extensions that would create edge repetitions (enforcing trails).

3.5 The Original LARPQ Algorithm for Reachability

We now present the original LARPQ algorithm for solving single-source 2-RPQ reachability, which serves as the foundation for our semiring generalization.

The algorithm maintains two matrices: M (the frontier) represents the relation between automaton states and graph vertices reached in the current step, while P accumulates all reachable pairs. The key insight is that on each iteration, $M_{qv} = 1$ if and only if there exists a 2-path in $\mathcal{G}$ from v_s to v and a path in $\mathcal{N}$ from some $q_s \in Q_S$ to q, both having the same labels and of length n (the iteration number). The mask $\langle \neg P \rangle$ ensures that each vertex-automaton state pair is processed only once, preventing infinite loops.

The correctness of the algorithm follows from the simultaneous breadth-first traversal of both the graph and the automaton. After termination, $P_{q_F v} = 1$ for $q_F \in Q_F$ if and only if there exists a 2-path from v_s to v whose label sequence belongs to the language accepted by $\mathcal{N}$.

The algorithm can also solve single-destination reachability by reversing the query. Let $\mathcal{R}^- = \{w^- = a_n^- a_{n-1}^- \ldots a_1^- \mid w = a_1 a_2 \ldots a_n \in \mathcal{R}\}$ and $\mathcal{N}^- = \langle Q, \Sigma, \Delta^-, \lambda_{\mathcal{N}}^-, Q_F, Q_S \rangle$. Then:

$$[[Q]]_{SDR} = [[\langle \mathcal{G}, \mathcal{R}^-, v_s \rangle]]_{SSR}.$$

Boolean matrix decomposition of $\mathcal{N}^-$ is obtained by transposing the matrices in the decomposition of $\mathcal{N}$.

4 A Semiring Framework for Single-Source RPQs

In this section, we present our main contribution: a generalized semiring-based framework for evaluating single-source 2-RPQs. The original LARPQ algorithm operates over the Boolean semiring to solve reachability. We show that by replacing the Boolean semiring with other algebraic structures, the same algorithmic framework can solve different path-aggregation problems.

4.1 Generalized Algorithm

The key insight is that the LARPQ algorithm can be abstracted to work over any semiring equipped with appropriate operations. Algorithm 2 presents the generalized version.

Input: Semiring $\mathcal{S} = \langle D_{out}, D_{in_1}, \mathbb{B}, \oplus, \otimes, \mathbf{0}\rangle$, optional index unary operator I, 2-NFA $\mathcal{N} = \langle Q, \Sigma, \Delta, \lambda_{\mathcal{N}}, Q_S, Q_F\rangle$, graph $\mathcal{G} = \langle V, E, L, \lambda_{\mathcal{G}}\rangle$, source $v_s \in V$
Output: Result depending on semiring instantiation

1. $\{N^a\} \leftarrow$ Boolean decomposition of the $\mathcal{N}$ adjacency matrix
2. $\{G^a\} \leftarrow$ Boolean decomposition of the $\mathcal{G}$ adjacency matrix
3. $M \leftarrow \mathbf{0}$, $P \leftarrow \mathbf{0}$
4. For each $q \in Q_S$: $M_{qv_s} \leftarrow \mathbf{1}$
5. While $M \neq \mathbf{0}$:
   - If I defined: $M \leftarrow I(M)$
   - $P \leftarrow P \oplus M$
   - $M \leftarrow \bigoplus_{a \in \Sigma^{\leftrightarrow} \cap L^{\leftrightarrow}} ((M^T \otimes N^a)^T \otimes G^a)$
6. Return Export results from P based on Q_F

Fig. 2: Generalized Single-Source 2-RPQ using Semirings

RS

maybe Return exported results? Fig. 3

The generalized algorithm differs from the original in several key aspects:

- Matrix operations ($\oplus$, $\otimes$) are performed over an arbitrary semiring $\mathcal{S}$ having $D_{in_2} = \mathbb{B}$;
- An optional index unary operator I can be applied to filter or transform path information;
- The final result format depends on the semiring instantiation.

We now present two important instantiations of this framework.

4.2 Instantiation I: Boolean Semiring for Reachability

By instantiating $\mathcal{S}$ with the Boolean semiring $\langle \mathbb{B} = \{0, 1\}, \mathbb{B}, \mathbb{B}, \vee, \wedge, 0\rangle$ and completely omitting the index unary operator, we recover the original LARPQ algorithm for reachability (Algorithm **??**). The matrix entries are Boolean values, where 1 indicates reachability. The mask operation $\langle \neg P \rangle$ ensures each vertex is processed only once

The result is a Boolean vector P^F where $P^F_v = 1$ if and only if vertex v is reachable from the source via a path satisfying the regular language constraint.

4.3 Instantiation II: Path Semiring with Index Operators for All Paths

For enumerating all simple paths or all trails, we use the previously introduced path semiring $\langle \mathrm{R}_{Paths}, \mathrm{R}_{Paths}, \mathbb{B} = \{0,1\}, \cup, \mathbf{fst}, \varnothing \rangle$ combined with appropriate index unary operators.

When $I = I_{Simple}$, the algorithm enumerates all simple 2-paths satisfying the query. When $I = I_{Trail}$, it enumerates all trails. The key difference from reachability is that the frontier matrix M now stores actual path information (as sequences of vertices) rather than Boolean values. The index unary operator filters extensions that would violate the path semantics:

- I_{Simple} prevents extending a path with a vertex already present in the path (except the starting vertex), ensuring simplicity;
- I_{Trail} prevents extending a path with an edge already traversed, ensuring the result is a trail.

The algorithm maintains the BFS structure: on iteration n, M_{qv} contains all valid paths of length n from v_s to v that correspond to paths from some $q_s \in Q_S$ to q in the automaton.

Figure 3 illustrates an example of the algorithm step for the all-paths semantics.

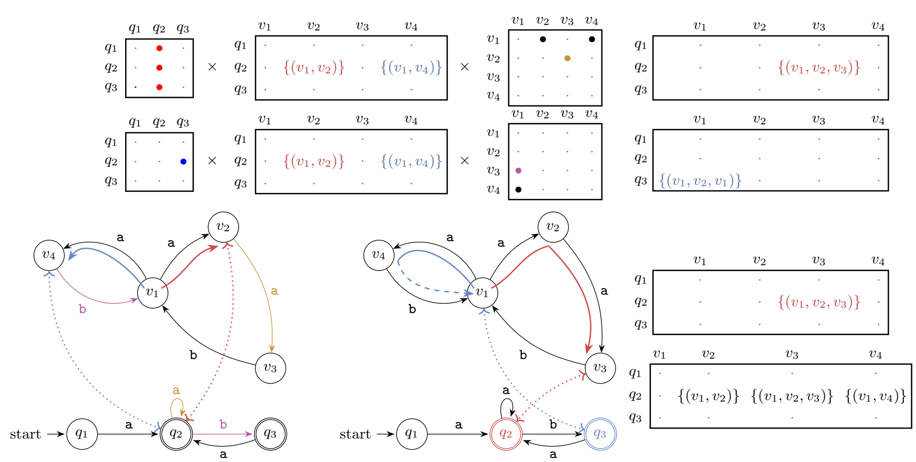


Fig. 3: The algorithm step for graph and automaton for regular expression $((a^+)(b?))^+$ using path semiring

4.4 Correctness Argument

The correctness argument follows the same induction scheme as in our previous work on LARPQ [6, Appendix A]. The induction is over the iterations of the main loop: the frontier matrix M stores exactly the paths produced at the current traversal depth, and the accumulator matrix P stores the union of all paths produced so far. Each iteration extends paths consistently in the query automaton and in the input graph; the optional index unary operator filters out extensions that violate the selected path semantics. After termination, selecting final automaton states yields exactly the query result for the chosen semiring instantiation.

5 Implementation Details

We have implemented the proposed algorithms within the LAGraph project [**?**], a collection of linear algebra-based graph processing algorithms built on top of SuiteSparse:GraphBLAS [**?**]. This section describes the key implementation aspects.

The actual implementation involves an additional semiring due to the asymmetric nature of the definition. The original semiring $\textsc{Paths} = \langle \mathrm{R}_{Paths}, \mathrm{R}_{Paths}, \mathbb{B} = \{0,1\}, \cup, \mathbf{fst}, \varnothing \rangle$ uses $\mathbf{fst} : \mathrm{R}_{Paths} \times \mathbb{B} \to \mathrm{R}_{Paths}$.

However, it is convenient to introduce another semiring $\overline{\textsc{Paths}} = \langle \mathrm{R}_{Paths}, \mathbb{B} = \{0,1\}, \mathrm{R}_{Paths}, \cup, \mathbf{snd}, \varnothing \rangle$, which uses $\mathbf{snd} : \mathbb{B} \times \mathrm{R}_{Paths} \to \mathrm{R}_{Paths}$, defined as ??.

If we use these two semirings and the corresponding multiplicative operations $\otimes$ and $\overline{\otimes}$, we obtain:

$$(A \otimes B)^T = (B^T \overline{\otimes} A^T)$$

for matrices A and B over D_{out}.

Using this observation, we can avoid the expensive product transposition on line ?? in Algorithm 2 and use the cheaper $(N^a)^T$ operation instead:

$$((M^T \otimes N^a)^T \otimes G^a) \rightsquigarrow ((N^a)^T \overline{\otimes} M \otimes G^a).$$

RS
should this dot be there?

In addition, SuiteSparse:GraphBLAS imposes restrictions on user-defined semirings. Most importantly, user-defined semirings must have fixed-size values to work with built-in automatic allocators. Therefore, the implementation of Paths in SuiteSparse:GraphBLAS relies on external memory management when paths become too long or when too many paths appear in a single M_{qv} R_{Paths} element. We introduce two constants, L_{fast} and C_{fast}, to control the maximum path length and the maximum cardinality of a R_{Paths} element without external memory management.

If a path $\pi \in M_{qv}$ with $|\pi| > L_{fast}$ appears, a new linked list representing this long path is allocated in a supplementary memory arena. Similarly, if $|M_{qv}| > C_{fast}$ for some q, v, an extensible linked list is allocated.

These two values provide a trade-off between peak memory consumption and execution speed. In practice, these constants should be adjusted for each dataset individually.

Finally, we ship the semiring-based algorithm implementation as a standalone executable file. A preprocessed graph, decomposed into a set of Boolean Matrix Market matrices, and a set of regular path query patterns described in a SPARQL-like query language are consumed as input and executed one by one. The CLI flag ?? `-semantics` can be used to switch between all-simple-path and all-trail semantics. At the same time, `-fast-path-length` controls L_{fast}, and `-fast-path-cardinality` defines the C_{fast} constant.

6 Evaluation

We evaluate the proposed solution on three semantics (simple paths, shortest paths, and trails) using real-world knowledge graphs. The experiments are designed to answer the following research questions.

- **RQ1:** How does our implementation compare with production graph database systems?
- **RQ2:** What is the performance behavior across different query types and graph structures?

Experiments are conducted on a workstation equipped with a Ryzen 9 7900X (4.7 GHz, 12 cores), 128 GB of DDR5 RAM, running Ubuntu 22.04. The timeout is set to 60 seconds per query. Each query is executed five times: one warm-up run followed by four measurement runs.

Competitors We compare the performance of the proposed semiring-based LARPQ algorithm against state-of-the-art graph database management systems.

- **MillenniumDB** or MDB (RPQ challenge version) [21]: a persistent, open-source graph database with state-of-the-art RPQ performance and natural support of all three path semantics.
- **FalkorDB** (v4.18.5, formerly RedisGraph [9]): an open-source in-memory property-graph database using GraphBLAS. Among the three path semantics, only all-trails is naturally expressed in FalkorDB, so we measure only the corresponding queries.

For all competitors, we use their respective query languages to specify queries. The time required for query processing (including parsing and optimization) is included in the measurements. All data is preloaded into storage, so graph preprocessing time is excluded from the results.

Dataset We used three different graphs and their respective sets of queries.

Wikidata [19]: 610 million edges, 91 million vertices, and 1 400 distinct labels. A total of 578 queries were filtered from a dump of 660 queries.

Yago-2S: 46 million edges, 7 million vertices, and 42 distinct labels. Seven complex queries were taken from [2].

RPQBench [22]: a synthetic dataset with 150 million edges, 57 million vertices, and 9 distinct labels. It includes 20 different query types, 10 queries were generated for each.

6.1 Experimental Results

We measure the total time for all queries, the mean and median time over all queries, and the mean and median per-query speedup over the proposed solution. A summary for all three graphs is presented in Tables 1, 3, and 2. Per-query results for Wikidata are shown in Figure 4. Only queries that were successfully evaluated for all competitors are included in these statistics. Figure 4d additionally shows queries completed by MillenniumDB and our solution but not by FalkorDB.

Table 1: Performance Summary on Wikidata (time in milliseconds)

Metric	Simple		Shortest		Trails		
	LARPQ	**MDB**	**LARPQ**	**MDB**	**LARPQ**	**MDB**	**FalkorDB**
Total time	**224 682.1**	1 111 429.4	**205 629.8**	780 011.1	152 128.6	921 506.4	**81 519.5**
Mean time	**428.8**	2 121.0	**392.4**	1 488.6	312.4	1 892.2	**167.4**
Median time	**1.7**	35.6	**2.0**	27.9	**1.4**	36.9	4.5
Mean Speedup	**1.0**	0.6	**1.0**	0.8	**1.0**	0.5	0.9
Median Speedup	**1.0**	0.1	**1.0**	0.1	**1.0**	0.1	0.7

Table 2: Performance Summary on RPQBench (time in milliseconds)

Metric	Simple		Shortest		Trails		
	LARPQ	**MDB**	**LARPQ**	**MDB**	**LARPQ**	**MDB**	**FalkorDB**
Total time	**857.7**	2835.7	**890.0**	3 253.3	**849.5**	3 256.5	1 685.3
Mean time	**42.9**	141.8	**44.5**	162.7	**53.1**	203.5	105.3
Median time	0.3	**0.2**	0.3	**0.1**	0.3	**0.2**	**0.2**
Mean Speedup	1.0	**1.5**	1.0	**2.1**	1.0	**1.5**	1.2
Median Speedup	1.0	**1.8**	1.0	**2.4**	1.0	**1.7**	1.5

The evaluation shows that execution time strongly depends on the particular query and the choice of start and target vertices. We define a query as *simple* for a system if it requires relatively little time to evaluate, and *hard* if, conversely, it requires a relatively large amount of time.

Table 3: Performance Summary on YAGO-2S (time in milliseconds)

Metric	Simple[1]		Shortest		Trails[1]		
	LARPQ	**MDB**	**LARPQ**	**MDB**	**LARPQ**	**MDB**	**FalkorDB**[2]
Total time	**104.8**	215.3	**1 194.6**	1 955.7	**66.4**	478.7	—
Mean time	—	—	**170.7**	279.4	—	—	—
Median time	—	—	**167.2**	268.3	—	—	—
Mean Speedup	—	—	**1.0**	0.5	—	—	—
Median Speedup	—	—	**1.0**	0.6	—	—	—

[1] Only two queries executed successfully for all competitors.
[2] Timeout for all queries.

For RPQBench (Table 2), the total and mean execution times of our solution are more than three times better than those of the competitors across all semantics. However, the median time and per-query speedup favor the competitors. This indicates that the dataset contains a relatively large number of simple queries, while our solution exhibits a significant advantage on harder queries.

This observation is consistent with the results presented in Figure 4, where MillenniumDB outperforms our solution on simple queries but is slower on difficult ones. The tail of queries that are challenging for MillenniumDB is thinner for LARPQ, as illustrated in Figure 4c. In the common-trails setting, FalkorDB achieves the best total and mean times, while LARPQ achieves the best median time.

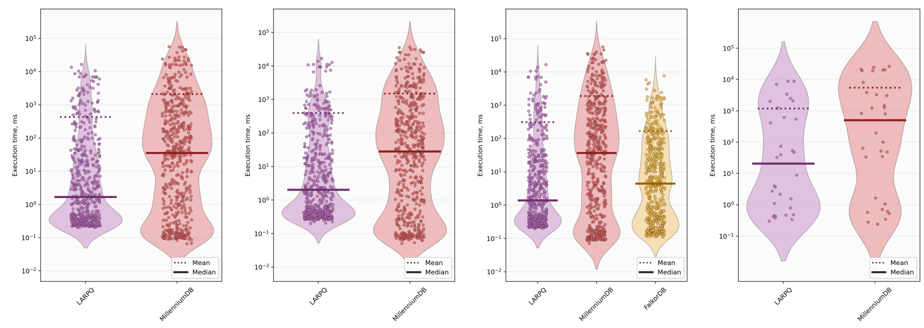
In the YAGO-2S dataset, the shortest-path semantics is the only one for which all seven queries completed successfully across all competing systems. For the simple-paths and trails semantics, many queries timed out. Nevertheless, for queries that completed successfully, our solution demonstrates better performance than the competitors.

A detailed analysis of which queries are *simple* and which are *hard* for our solution yields results similar to those reported in [6], indicating that the proposed generalization significantly inherits the properties of the basic version. Specifically, LARPQ outperforms competitors on queries with complex patterns over labels corresponding to many edges (e.g., $l_1/l_2/(l_3 \mid l_4)^*$), while on queries with simple patterns such as l_1^*/l_2, it shows no performance advantage, and MillenniumDB may be a better choice.

Overall, the results indicate that LARPQ is a strong choice for complex RPQs, especially when query patterns involve labels with many matching edges, whereas competing systems may perform better on simpler patterns.

7 Related Work

The algebraic approach to path problems has a long history. Kleene's algorithm computes reachability using matrix operations over the Boolean semiring. This approach has been generalized to arbitrary semirings for solving various algebraic path problems [4], including path enumeration. Different query semantics and



(a) Simple paths (b) Shortest paths (c) Trails (common for all competitors) (d) Trails (FalkorDB failed)

Fig. 4: Performance on Wikidata

respective algorithms for regular path queries, including all shortest paths and all simple paths, have been studied in the literature [16, 17, 20].

The GraphBLAS initiative [15, 8, 12] established a standardized interface for expressing graph algorithms using linear algebra operations. The GraphBLAS specification supports arbitrary semirings, enabling the expression of algorithms that go beyond traditional numeric computations. LAGraph [18] provides a collection of graph algorithms built on GraphBLAS, including BFS, PageRank, and triangle counting. Recent work has explored linear algebra approaches for various graph problems. However, RPQ evaluation in this paradigm remains underexplored, with only a few prior works [3, 6] addressing this problem.

8 Conclusion and Future Work

We have presented a semiring-based generalization of the linear algebra approach for evaluating two-way regular path queries. Source code and additional materials are available at https://github.com/SparseLinearAlgebra/la-rpq. Experimental evaluation demonstrates that the proposed approach achieves competitive performance with state-of-the-art graph database systems while offering a more flexible framework for different path semantics. At the same time, our evaluation reveals limitations of both the proposed and existing solutions in terms of performance (see the discussion of results on YAGO-2S 3), opening the door for further research on performance improvements.

Future work includes the use of additional semirings, such as counting semirings for path counting, as well as integration with property graphs and more expressive query languages such as CRPQs. Another direction is distributed implementation and acceleration through offloading matrix operations to GPUs.

Acknowledgments. This research has been supported by the St. Petersburg State University, grant number 116636233.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. SPARQL 1.1 Query Language. Tech. rep., W3C (2013), http://www.w3.org/TR/sparql11-query
2. Abul-Basher, Z., Yakovets, N., Godfrey, P., Ghajar-Khosravi, S., Chignell, M.H.: TASWEET: optimizing disjunctive path queries in graph databases. In: Markl, V., Orlando, S., Mitschang, B., Andritsos, P., Sattler, K., Breß, S. (eds.) Proceedings of the 20th International Conference on Extending Database Technology, EDBT 2017, Venice, Italy, March 21-24, 2017. pp. 470–473. OpenProceedings.org (2017). https://doi.org/10.5441/002/EDBT.2017.47, https://doi.org/10.5441/002/edbt.2017.47
3. Arroyuelo, D., Gómez-Brandón, A., Navarro, G.: Evaluating regular path queries on compressed adjacency matrices. In: String Processing and Information Retrieval: 30th International Symposium, SPIRE 2023, Pisa, Italy, September 26–28, 2023, Proceedings. p. 35–48. Springer-Verlag, Berlin, Heidelberg (2023). https://doi.org/10.1007/978-3-031-43980-3_4, https://doi.org/10.1007/978-3-031-43980-3_4
4. Baras, J.S., Theodorakopoulos, G.: Path problems in networks. Synthesis Lectures on Communication Networks **3**, 1–77 (2010), https://api.semanticscholar.org/CorpusID:38871641
5. Barrett, C., Jacob, R., Marathe, M.: Formal-language-constrained path problems. SIAM Journal on Computing **30**(3), 809–837 (2000). https://doi.org/10.1137/S0097539798337716, https://doi.org/10.1137/S0097539798337716
6. Belyanin, G., Suvorov, R., Grigorev, S.: Single-source regular path querying in terms of linear algebra. Proceedings of the VLDB Endowment. ISSN **2150**, 8097
7. Bienvenu, M., Calvanese, D., Ortiz, M., Simkus, M.: Nested regular path queries in description logics (extended abstract). In: Gottlob, G., Pérez, J. (eds.) Proceedings of the 8th Alberto Mendelzon Workshop on Foundations of Data Management, Cartagena de Indias, Colombia, June 4-6, 2014. CEUR Workshop Proceedings, CEUR-WS.org (2014), https://ceur-ws.org/Vol-1189/paper_34.pdf
8. Brock, B., Buluç, A., Mattson, T., McMillan, S., Moreira, J.: The graphblas c api specification. GraphBLAS. org, Tech. Rep (2021)
9. Cailliau, P., Davis, T., Gadepally, V., Kepner, J., Lipman, R., Lovitz, J., Ouaknine, K.: Redisgraph graphblas enabled graph database. In: 2019 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW). p. 285–286. IEEE (May 2019). https://doi.org/10.1109/ipdpsw.2019.00054, http://dx.doi.org/10.1109/IPDPSW.2019.00054
10. Calvanese, D., De Giacomo, G., Lenzerini, M., Vardi, M.Y.: Reasoning on regular path queries. SIGMOD Record **32**(4), 83–92 (2003). https://doi.org/10.1145/959060.959076
11. Casel, K., Schmid, M.L.: Fine-Grained Complexity of Regular Path Queries. In: Yi, K., Wei, Z. (eds.) 24th International Conference on Database Theory (ICDT 2021). Leibniz International Proceedings in Informatics (LIPIcs), vol. 186, pp. 19:1–19:20. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2021). https://doi.org/10.4230/LIPIcs.ICDT.2021.19, https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ICDT.2021.19

12. Davis, T.A.: Algorithm 1037: Suitesparse:graphblas: Parallel graph algorithms in the language of sparse linear algebra. ACM Trans. Math. Softw. **49**(3) (Sep 2023). https://doi.org/10.1145/3577195, https://doi.org/10.1145/3577195
13. Farias, B., Rojas, C., Vrgoc, D.: Millenniumdb path query challenge (short paper). In: Kimelfeld, B., Martinez, M.V., Angles, R. (eds.) Proceedings of the 15th Alberto Mendelzon International Workshop on Foundations of Data Management (AMW 2023), Santiago de Chile, Chile, May 22-26, 2023. CEUR Workshop Proceedings, CEUR-WS.org (2023), https://ceur-ws.org/Vol-3409/paper13.pdf
14. Information technology – Database languages – GQL. Standard, International Organization for Standardization, Geneva, CH (2024), https://www.iso.org/standard/76120.html
15. Kepner, J., Aaltonen, P., Bader, D.A., Buluç, A., Franchetti, F., Gilbert, J.R., Hutchison, D., Kumar, M., Lumsdaine, A., Meyerhenke, H., McMillan, S., Yang, C., Owens, J.D., Zalewski, M., Mattson, T.G., Moreira, J.E.: Mathematical foundations of the graphblas. 2016 IEEE High Performance Extreme Computing Conference (HPEC) pp. 1–9 (2016), https://api.semanticscholar.org/CorpusID:3654505
16. Martens, W., Trautner, T.: Evaluation and Enumeration Problems for Regular Path Queries. In: Kimelfeld, B., Amsterdamer, Y. (eds.) 21st International Conference on Database Theory (ICDT 2018). Leibniz International Proceedings in Informatics (LIPIcs), vol. 98, pp. 19:1–19:21. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2018). https://doi.org/10.4230/LIPIcs.ICDT.2018.19, https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ICDT.2018.19
17. Mendelzon, A.O., Wood, P.T.: Finding regular simple paths in graph databases. SIAM J. Comput. **24**, 1235–1258 (1989), https://api.semanticscholar.org/CorpusID:12684556
18. Szárnyas, G., Bader, D.A., Davis, T.A., Kitchen, J., Mattson, T.G., McMillan, S., Welch, E.: Lagraph: Linear algebra, network analysis libraries, and the study of graph algorithms (2021), https://arxiv.org/abs/2104.01661
19. Vrandečić, D., Krötzsch, M.: Wikidata: a free collaborative knowledgebase. Commun. ACM **57**(10), 78–85 (sep 2014). https://doi.org/10.1145/2629489, https://doi.org/10.1145/2629489
20. Vrgoc, D.: Evaluating regular path queries under the all-shortest paths semantics. CoRR **abs/2204.11137** (2022). https://doi.org/10.48550/ARXIV.2204.11137, https://doi.org/10.48550/arXiv.2204.11137
21. Vrgoc, D., Rojas, C., Angles, R., Arenas, M., Arroyuelo, D., Aranda, C.B., Hogan, A., Navarro, G., Riveros, C., Romero, J.: Millenniumdb: A persistent, open-source, graph database (2021), https://arxiv.org/abs/2111.01540
22. Wang, H., Wang, X., Ma, M., You, Y.: Rpqbench: A benchmark for regular path queries on graph data. In: Barhamgi, M., Wang, H., Wang, X. (eds.) Web Information Systems Engineering – WISE 2024. pp. 351–367. Springer Nature Singapore, Singapore (2025)